



SIMATS ENGINEERING

**Saveetha Institute of Medical and
Technical Sciences Chennai-602105**



Engineer to Excel

**Civic Track: Crowdsourced Civic Issue Reporting and Resolution
System**

CSA0804 – Python Programming

Submitted by

Perarasan V (192521404)

PSEUDOCODE

START

CREATE empty dictionary water_records

FUNCTION classify_consumption(consumption)

IF consumption < 200

 RETURN "Low Consumption"

ELSE IF consumption <= 500

 RETURN "Moderate Consumption"

ELSE

 RETURN "High Consumption"

END FUNCTION

FUNCTION clear_screen

 Remove all widgets from the current window

END FUNCTION

FUNCTION home_page

 Clear screen

 Display application title

Display buttons:

Add Record

Display Records

Search Record

Update Record

Water Analysis

High Consumption

Zone-wise Analysis

Unique Data Analysis

Exit

END FUNCTION

FUNCTION add_record_page

Clear screen

Display input fields:

Household ID

Zone

Water Supplied

Water Consumption

Date

WHEN user clicks SAVE RECORD

Get all values from input fields

IF any field is empty

Display error message

ELSE

Convert supplied and consumption into numbers

IF values are invalid

Display error message

ELSE

category = classify_consumption(consumption)

Store record in water_records dictionary

Display "Record Saved Successfully"

END IF

END IF

END FUNCTION

FUNCTION display_records

Clear screen

IF water_records is empty

Display "No Records Available"

ELSE

FOR each household ID and record

Display all water details

END FOR

END IF

END FUNCTION

FUNCTION search_record

Get Household ID from user

IF Household ID exists in water_records

Display the record details

ELSE

Display "Record Not Found"

END IF

END FUNCTION

FUNCTION update_record

Get Household ID

IF Household ID does not exist

Display "Record Not Found"

ELSE

Get new consumption value

Update consumption

Recalculate consumption category

Display success message

END IF

END FUNCTION

FUNCTION water_analysis

total_supplied = 0

total_consumption = 0

FOR each record in water_records

Add supplied water to total_supplied

Add consumption to total_consumption

END FOR

```
remaining_water =  
    total_supplied - total_consumption
```

```
average_consumption =  
    total_consumption / number of records
```

```
consumption_percentage =  
    total_consumption / total_supplied * 100
```

```
Display all analysis results
```

```
END FUNCTION
```

```
FUNCTION excessive_consumption
```

```
    Get consumption limit
```

```
    FOR each record
```

```
        IF consumption is greater than limit
```

```
            Display Household ID
```

```
            Display Zone
```

```
            Display Consumption
```

```
        END IF
```

```
    END FOR
```

```
END FUNCTION
```

FUNCTION zone_analysis

 CREATE empty dictionary zone_data

 FOR each record

 Get zone

 Add consumption to that zone

 END FOR

 Display total consumption of every zone

END FUNCTION

FUNCTION unique_data_analysis

 CREATE empty sets:

 unique_zones

 unique_users

 unique_categories

 FOR each record

 Add zone to unique_zones

 Add Household ID to unique_users

Add category to unique_categories

END FOR

Display all unique values

END FUNCTION

CREATE Tkinter main window

Set window title

Set window size

Call home_page

Start Tkinter mainloop

END

```
import tkinter as tk
from tkinter import messagebox

# =====
# MAIN DICTIONARY
# =====

water_records = {}

# =====
# CONSUMPTION CLASSIFICATION
# =====

def classify_consumption(consumption):

    if consumption < 200:
        return "Low Consumption"

    elif consumption <= 500:
        return "Moderate Consumption"

    else:
        return "High Consumption"

# =====
# CLEAR SCREEN
# =====

def clear_screen():

    for widget in root.winfo_children():
        widget.destroy()
```

```
# =====  
# HOME PAGE  
# =====
```

```
def home_page():
```

```
    clear_screen()
```

```
    title = tk.Label(  
        root,  
        text="SMART WATER MANAGEMENT SYSTEM",  
        font=("Arial", 18, "bold")  
    )
```

```
    title.pack(pady=25)
```

```
    subtitle = tk.Label(  
        root,  
        text="Water Distribution and Consumption Analytics",  
        font=("Arial", 11)  
    )
```

```
    subtitle.pack(pady=5)
```

```
# BUTTON FRAME
```

```
button_frame = tk.Frame(root)
```

```
button_frame.pack(pady=20)
```

```
tk.Button(  
    )
```

```
button_frame,  
text="ADD RECORD",  
width=25,  
height=2,  
command=add_record_page  
)grid(row=0, column=0, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="DISPLAY RECORDS",  
    width=25,  
    height=2,  
    command=display_records  
)grid(row=0, column=1, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="SEARCH RECORD",  
    width=25,  
    height=2,  
    command=search_record_page  
)grid(row=1, column=0, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="UPDATE RECORD",  
    width=25,  
    height=2,  
    command=update_record_page  
)grid(row=1, column=1, padx=10, pady=10)
```

```
tk.Button(  

```

```
button_frame,  
text="WATER ANALYSIS",  
width=25,  
height=2,  
command=water_analysis  
)grid(row=2, column=0, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="HIGH CONSUMPTION",  
    width=25,  
    height=2,  
    command=excessive_consumption_page  
)grid(row=2, column=1, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="ZONE-WISE ANALYSIS",  
    width=25,  
    height=2,  
    command=zone_analysis  
)grid(row=3, column=0, padx=10, pady=10)
```

```
tk.Button(  
    button_frame,  
    text="UNIQUE DATA ANALYSIS",  
    width=25,  
    height=2,  
    command=unique_data_analysis  
)grid(row=3, column=1, padx=10, pady=10)
```

```
tk.Button(  

```

```
    root,  
    text="EXIT",  
    width=20,  
    height=2,  
    command=root.destroy  
) .pack(pady=15)
```

```
# =====
```

```
# ADD RECORD PAGE
```

```
# =====
```

```
def add_record_page():
```

```
    clear_screen()
```

```
    tk.Label(  
        root,  
        text="ADD WATER RECORD",  
        font=("Arial", 18, "bold")  
    ).pack(pady=20)
```

```
    form = tk.Frame(root)
```

```
    form.pack(pady=10)
```

```
# HOUSEHOLD ID
```

```
    tk.Label(  
        form,  
        text="Household ID"  
    ).grid(row=0, column=0, padx=10, pady=10)
```

```
id_entry = tk.Entry(  
    form,  
    width=30  
)
```

```
id_entry.grid(row=0, column=1)
```

```
# ZONE
```

```
tk.Label(  
    form,  
    text="Zone"  
)grid(row=1, column=0, padx=10, pady=10)
```

```
zone_entry = tk.Entry(  
    form,  
    width=30  
)
```

```
zone_entry.grid(row=1, column=1)
```

```
# WATER SUPPLIED
```

```
tk.Label(  
    form,  
    text="Water Supplied (Litres)"  
)grid(row=2, column=0, padx=10, pady=10)
```

```
supplied_entry = tk.Entry(  
    form,  
    width=30  
)
```

```
supplied_entry.grid(row=2, column=1)
```

```
# WATER CONSUMPTION
```

```
tk.Label(  
    form,  
    text="Water Consumption (Litres)"  
).grid(row=3, column=0, padx=10, pady=10)
```

```
consumption_entry = tk.Entry(  
    form,  
    width=30  
)
```

```
consumption_entry.grid(row=3, column=1)
```

```
# DATE
```

```
tk.Label(  
    form,  
    text="Date"  
).grid(row=4, column=0, padx=10, pady=10)
```

```
date_entry = tk.Entry(  
    form,  
    width=30  
)
```

```
date_entry.grid(row=4, column=1)
```

```
# =====
```



```
# SAVE RECORD
```

```
# =====
```

```
def save_record():
```

```
    household_id = id_entry.get().strip().upper()
```

```
    zone = zone_entry.get().strip().title()
```

```
    supplied = supplied_entry.get().strip()
```

```
    consumption = consumption_entry.get().strip()
```

```
    date = date_entry.get().strip()
```

```
# EMPTY CHECK
```

```
if (
```

```
    household_id == ""
```

```
    or zone == ""
```

```
    or supplied == ""
```

```
    or consumption == ""
```

```
    or date == ""
```

```
):
```

```
    messagebox.showerror(
```

```
        "Error",
```

```
        "Please fill all fields"
```

```
    )
```

```
    return
```

```
# NUMBER VALIDATION
```

```
try:
```

```
    supplied = float(supplied)
```

```
    consumption = float(consumption)
```

```
except ValueError:
```

```
    messagebox.showerror(  
        "Error",  
        "Water values must be numbers"  
    )
```

```
    return
```

```
# NEGATIVE VALUE CHECK
```

```
if supplied < 0 or consumption < 0:
```

```
    messagebox.showerror(  
        "Error",  
        "Values cannot be negative"  
    )
```

```
    return
```

```
# CLASSIFICATION
```

```
category = classify_consumption(  
    consumption
```

)

=====

STORE IN DICTIONARY

=====

water_records[household_id] = {

"zone": zone,

"supplied": supplied,

"consumption": consumption,

"date": date,

"category": category

}

messagebox.showinfo(

"Success",

"Record Saved Successfully!"

)

CLEAR INPUT BOXES

id_entry.delete(0, tk.END)

zone_entry.delete(0, tk.END)

supplied_entry.delete(0, tk.END)

```
consumption_entry.delete(0, tk.END)
```

```
date_entry.delete(0, tk.END)
```

```
# SAVE BUTTON
```

```
tk.Button(  
    root,  
    text="SAVE RECORD",  
    width=20,  
    height=2,  
    command=save_record  
).pack(pady=15)
```

```
tk.Button(  
    root,  
    text="BACK TO MENU",  
    width=20,  
    command=home_page  
).pack()
```

```
# =====
```

```
# DISPLAY RECORDS
```

```
# =====
```

```
def display_records():
```

```
    clear_screen()
```

```
tk.Label(  
    root,  
    text="ALL WATER RECORDS",
```

```
font=("Arial", 18, "bold")
).pack(pady=20)
```

```
result = tk.Text(
    root,
    width=70,
    height=25
)
```

```
result.pack(pady=10)
```

```
if len(water_records) == 0:
```

```
    result.insert(
        tk.END,
        "No Records Available"
    )
```

```
else:
```

```
    for household_id, record in water_records.items():
```

```
        result.insert(
            tk.END,

            f"Household ID: {household_id}\n"
            f"Zone: {record['zone']}\n"
            f"Water Supplied: {record['supplied']} Litres\n"
            f"Water Consumption: {record['consumption']} Litres\n"
            f>Date: {record['date']}\n"
            f"Category: {record['category']}\n"
            f"{'-' * 45}\n\n"
```

)

```
tk.Button(  
    root,  
    text="BACK TO MENU",  
    width=20,  
    command=home_page  
).pack(pady=10)
```

```
# =====  
# SEARCH RECORD  
# =====
```

```
def search_record_page():
```

```
    clear_screen()
```

```
    tk.Label(  
        root,  
        text="SEARCH WATER RECORD",  
        font=("Arial", 18, "bold")  
    ).pack(pady=25)
```

```
    tk.Label(  
        root,  
        text="Enter Household ID"  
    ).pack()
```

```
    search_entry = tk.Entry(  
        root,  
        width=30  
    )
```

```
search_entry.pack(pady=10)
```

```
result = tk.Text(  
    root,  
    width=60,  
    height=12  
)
```

```
result.pack(pady=10)
```

```
def search_record():
```

```
    household_id = (  
        search_entry.get()  
        .strip()  
        .upper()  
    )
```

```
    result.delete(  
        "1.0",  
        tk.END  
    )
```

```
    if household_id in water_records:
```

```
        record = water_records[  
            household_id  
        ]
```

```
        result.insert(
```

```

        tk.END,

        f"Household ID: {household_id}\n\n"
        f"Zone: {record['zone']}\n"
        f"Water Supplied: {record['supplied']} Litres\n"
        f"Water Consumption: {record['consumption']} Litres\n"
        f>Date: {record['date']}\n"
        f"Category: {record['category']}"
    )

else:

    result.insert(
        tk.END,
        "Record Not Found!"
    )

tk.Button(
    root,
    text="SEARCH",
    width=20,
    command=search_record
).pack(pady=10)

tk.Button(
    root,
    text="BACK TO MENU",
    width=20,
    command=home_page
).pack()

```

```
# =====
```



```
# UPDATE RECORD
```

```
# =====
```

```
def update_record_page():
```

```
    clear_screen()
```

```
    tk.Label(
        root,
        text="UPDATE WATER RECORD",
        font=("Arial", 18, "bold")
    ).pack(pady=25)
```

```
    tk.Label(
        root,
        text="Household ID"
    ).pack()
```

```
    id_entry = tk.Entry(
        root,
        width=30
    )
```

```
    id_entry.pack(pady=5)
```

```
    tk.Label(
        root,
        text="New Water Consumption"
    ).pack()
```

```
    consumption_entry = tk.Entry(
        root,
```

```

        width=30
    )

consumption_entry.pack(pady=5)

def update_record():

    household_id = (
        id_entry.get()
        .strip()
        .upper()
    )

    if household_id not in water_records:

        messagebox.showerror(
            "Error",
            "Record Not Found"
        )

    return

try:

    consumption = float(
        consumption_entry.get()
    )

except ValueError:

    messagebox.showerror(
        "Error",

```

```

        "Enter a valid number"
    )

    return

water_records[
    household_id
]["consumption"] = consumption

water_records[
    household_id
]["category"] = (
    classify_consumption(
        consumption
    )
)

messagebox.showinfo(
    "Success",
    "Record Updated Successfully!"
)

tk.Button(
    root,
    text="UPDATE RECORD",
    width=20,
    command=update_record
).pack(pady=15)

tk.Button(
    root,
    text="BACK TO MENU",

```

```

        width=20,
        command=home_page
    ).pack()

# =====
# WATER ANALYSIS
# =====

def water_analysis():

    clear_screen()

    tk.Label(
        root,
        text="WATER ANALYSIS",
        font=("Arial", 18, "bold")
    ).pack(pady=25)

    result = tk.Text(
        root,
        width=60,
        height=18,
        font=("Arial", 12)
    )

    result.pack(pady=10)

    if len(water_records) == 0:

        result.insert(
            tk.END,
            "No Records Available"

```

)

else:

total_supplied = 0

total_consumption = 0

for record in water_records.values():

total_supplied += (
 record["supplied"]
)

total_consumption += (
 record["consumption"]
)

remaining_water = (
 total_supplied
 - total_consumption
)

average_consumption = (
 total_consumption
 / len(water_records)
)

if total_supplied > 0:

percentage = (
 total_consumption

```
        / total_supplied  
    ) * 100
```

else:

```
    percentage = 0
```

```
result.insert(
```

```
    tk.END,
```

```
    "WATER ANALYTICAL REPORT\n"
```

```
    "=====\n\n"
```

```
    f"Total Water Supplied: "
```

```
    f"{total_supplied} Litres\n\n"
```

```
    f"Total Water Consumed: "
```

```
    f"{total_consumption} Litres\n\n"
```

```
    f"Remaining Water: "
```

```
    f"{remaining_water} Litres\n\n"
```

```
    f"Average Consumption: "
```

```
    f"{average_consumption:.2f} Litres\n\n"
```

```
    f"Consumption Percentage: "
```

```
    f"{percentage:.2f}%"
```

```
)
```

```
tk.Button(
```

```
    root,  
    text="BACK TO MENU",  
    width=20,  
    command=home_page  
).pack(pady=15)
```

```
# =====  
# EXCESSIVE CONSUMPTION  
# =====
```

```
def excessive_consumption_page():
```

```
    clear_screen()
```

```
    tk.Label(  
        root,  
        text="EXCESSIVE CONSUMPTION",  
        font=("Arial", 18, "bold")  
    ).pack(pady=25)
```

```
    tk.Label(  
        root,  
        text="Enter Consumption Limit"  
    ).pack()
```

```
    limit_entry = tk.Entry(  
        root,  
        width=30  
    )
```

```
    limit_entry.pack(pady=10)
```

```
result = tk.Text(  
    root,  
    width=60,  
    height=15  
)
```

```
result.pack(pady=10)
```

```
def check_consumption():
```

```
    result.delete(  
        "1.0",  
        tk.END  
    )
```

```
    try:
```

```
        limit = float(  
            limit_entry.get()  
        )
```

```
    except ValueError:
```

```
        result.insert(  
            tk.END,  
            "Enter a valid number"  
        )
```

```
    return
```

```
found = False
```



```
for household_id, record in water_records.items():
```

```
    if (
```

```
        record["consumption"]
```

```
        > limit
```

```
    ):

```

```
        result.insert(
```

```
            tk.END,
```

```
            f"ID: {household_id}\n"
```

```
            f"Zone: {record['zone']}\n"
```

```
            f"Consumption: "
```

```
            f"{record['consumption']} Litres\n\n"
```

```
        )

```

```
        found = True

```

```
if found == False:
```

```
    result.insert(
```

```
        tk.END,
```

```
        "No Excessive Consumption Found"
```

```
    )

```

```
tk.Button(
```

```
    root,
```

```
    text="CHECK",
```

```
    width=20,
```

```
        command=check_consumption
    ).pack(pady=10)
```

```
tk.Button(
    root,
    text="BACK TO MENU",
    width=20,
    command=home_page
).pack()
```

```
# =====
# ZONE-WISE ANALYSIS
# =====
```

```
def zone_analysis():
```

```
    clear_screen()
```

```
tk.Label(
    root,
    text="ZONE-WISE ANALYSIS",
    font=("Arial", 18, "bold")
).pack(pady=25)
```

```
result = tk.Text(
    root,
    width=60,
    height=20
)
```

```
result.pack(pady=10)
```

```
zone_data = {}

for record in water_records.values():

    zone = record["zone"]

    consumption = (
        record["consumption"]
    )

    if zone in zone_data:

        zone_data[zone] += consumption

    else:

        zone_data[zone] = consumption

if len(zone_data) == 0:

    result.insert(
        tk.END,
        "No Records Available"
    )

else:

    for zone, consumption in zone_data.items():

        result.insert(

            tk.END,
```

```
f"Zone: {zone}\n"
```

```
f"Total Consumption: "
```

```
f"{consumption} Litres\n"
```

```
f"{'-' * 35}\n"
```

```
)
```

```
tk.Button(
```

```
    root,
```

```
    text="BACK TO MENU",
```

```
    width=20,
```

```
    command=home_page
```

```
).pack(pady=10)
```

```
# =====
```

```
# UNIQUE DATA ANALYSIS
```

```
# =====
```

```
def unique_data_analysis():
```

```
    clear_screen()
```

```
    tk.Label(
```

```
        root,
```

```
        text="UNIQUE DATA ANALYSIS",
```

```
        font=("Arial", 18, "bold")
```

```
).pack(pady=25)
```

```
result = tk.Text(
```

```
    root,
```

```

        width=60,
        height=18
    )

    result.pack(pady=10)

    unique_zones = set()

    unique_users = set()

    unique_categories = set()

    for household_id, record in water_records.items():

        unique_users.add(
            household_id
        )

        unique_zones.add(
            record["zone"]
        )

        unique_categories.add(
            record["category"]
        )

    result.insert(

        tk.END,

        "UNIQUE ZONES\n"
        "=====\n"

```

```
f"{unique_zones}\n\n"
```

```
"UNIQUE HOUSEHOLD IDs\n"
```

```
"=====\n"
```

```
f"{unique_users}\n\n"
```

```
"CONSUMPTION CATEGORIES\n"
```

```
"=====\n"
```

```
f"{unique_categories}"
```

```
)
```

```
tk.Button(
```

```
    root,
```

```
    text="BACK TO MENU",
```

```
    width=20,
```

```
    command=home_page
```

```
).pack(pady=10)
```

```
# =====
```

```
# CREATE MAIN WINDOW
```

```
# =====
```

```
root = tk.Tk()
```

```
root.title(
```

```
    "Smart Water Management System"
```

```
)
```

```
root.geometry("750x650")
```

```
# START HOME PAGE
```

```
home_page()
```

```
# KEEP APPLICATION RUNNING
```

```
root.mainloop()
```

